Práctica 3

Circuitos Combinacionales

Decodificador BCD y Display de 7 Segmentos

Integrantes:
Yeremy Vega Pineda - 20242020233

Fecha: 17 de abril de 2026

1. Introducción

En esta práctica se trabaja con el diseño de una Unidad Aritmético-Lógica (ALU), que es una de las partes más importantes en cualquier sistema digital, especialmente en los computadores. La ALU es la encargada de realizar operaciones básicas como sumas, restas y operaciones lógicas entre datos binarios.

El propósito de esta actividad es entender cómo funciona una ALU desde cero, construyéndola únicamente con compuertas lógicas y circuitos combinacionales. Esto permite ver de forma más clara cómo se pueden implementar operaciones complejas a partir de elementos simples.

Un circuito combinacional es aquel cuyas salidas dependen únicamente de las entradas actuales, sin tener en cuenta estados anteriores. Este tipo de circuitos es la base para construir sistemas como la ALU, ya que permiten implementar funciones matemáticas y lógicas usando compuertas como AND, OR, NOT y XOR.

Las operaciones aritmético-lógicas son fundamentales en el procesamiento de datos. Por un lado, están las operaciones aritméticas como la suma, resta y multiplicación, que trabajan directamente con valores numéricos. Por otro lado, están las operaciones lógicas, que comparan o transforman bits de información.

En esta práctica, se integran todos estos conceptos para construir una ALU de 8 bits completamente funcional. Además, se utiliza un multiplexor para seleccionar qué operación se desea ejecutar en cada momento, lo que permite simular el comportamiento básico de una ALU real.

2. Descripción del desarrollo

En esta práctica se construyó una ALU de 8 bits paso a paso, diseñando primero cada uno de sus componentes por separado y luego integrándolos en un solo circuito en Logisim. Todo se hizo usando únicamente compuertas lógicas básicas.

2.1. Operadores lógicos

Las operaciones lógicas se implementaron de forma bit a bit entre las entradas A y B. Para cada operación se usaron 8 compuertas en paralelo, una por cada bit.

Debido a que estas operaciones son directas, no requieren un análisis tan complejo como las aritméticas. Sin embargo, se muestran sus implementaciones para mayor claridad.

2.1.1. AND

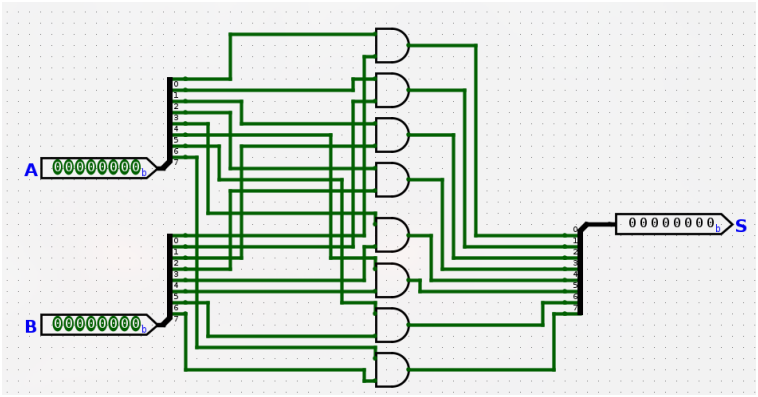

Figura 1: Implementación de la operación AND

2.1.2. OR

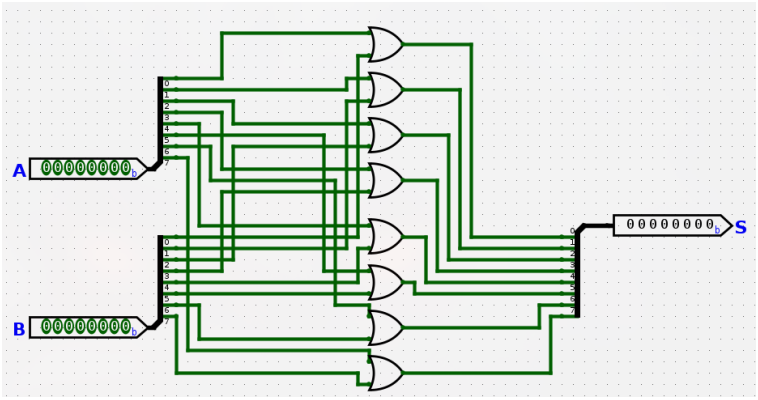

Figura 2: Implementación de la operación OR

2.1.3. XOR

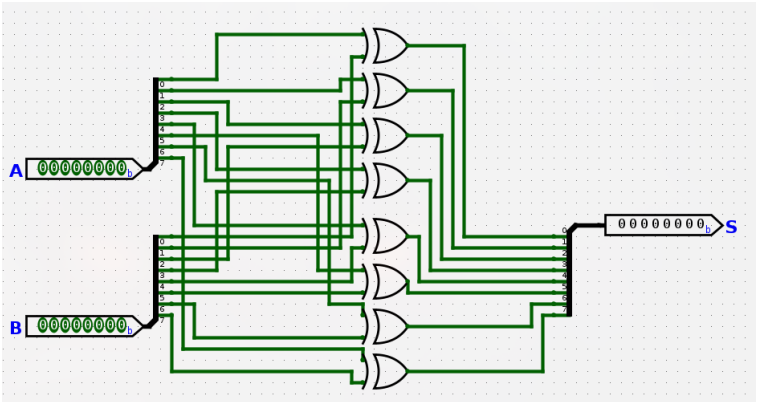

Figura 3: Implementación de la operación XOR

2.1.4. NOT

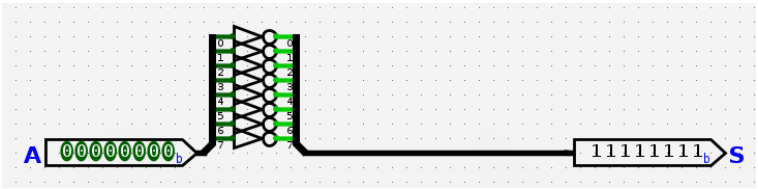

Figura 4: Implementación de la operación NOT (solo sobre A)

2.1.5. XNOR

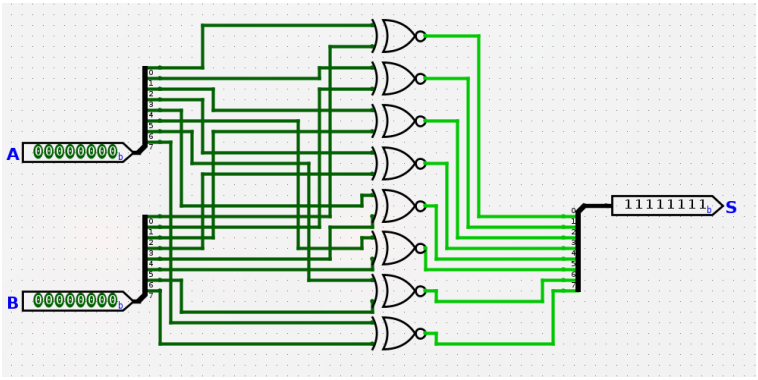

Figura 5: Implementación de la operación XNOR

Cada uno de estos bloques fue probado de forma independiente con varios valores de entrada para verificar su funcionamiento.

2.2. Sumador de 8 bits

El diseño del sumador se realizó de forma progresiva, comenzando desde un sumador de 1 bit y escalando hasta llegar a 8 bits.

2.2.1. Sumador de 1 bit (Full Adder)

Primero se construyó un sumador completo de 1 bit utilizando compuertas lógicas. Este recibe tres entradas: A, B y el acarreo de entrada (Cin), y genera dos salidas: la suma (S) y el acarreo de salida (Cout).

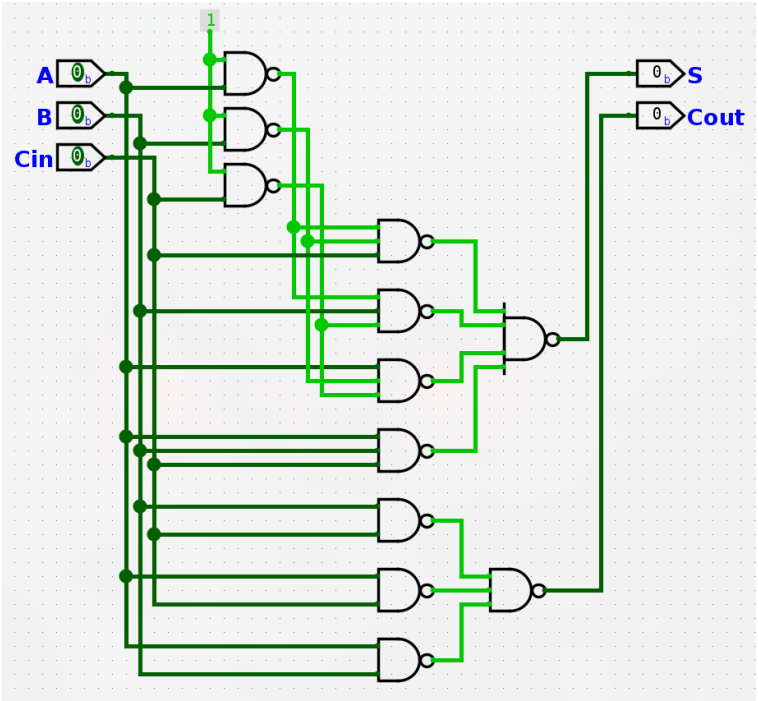

Figura 6: Sumador completo de 1 bit

2.2.2. Sumador de 2 bits

Luego se conectaron dos sumadores de 1 bit en cascada. El acarreo de salida del primer bloque se conecta como entrada del segundo.

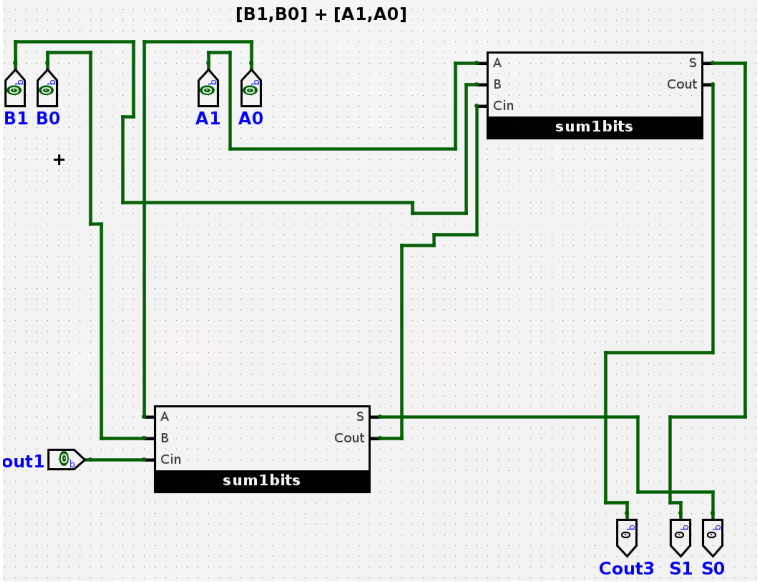

Figura 7: Sumador de 2 bits

2.2.3. Sumador de 4 bits

Siguiendo la misma idea, se construyó un sumador de 4 bits utilizando dos sumadores de 2 bits conectados en cadena.

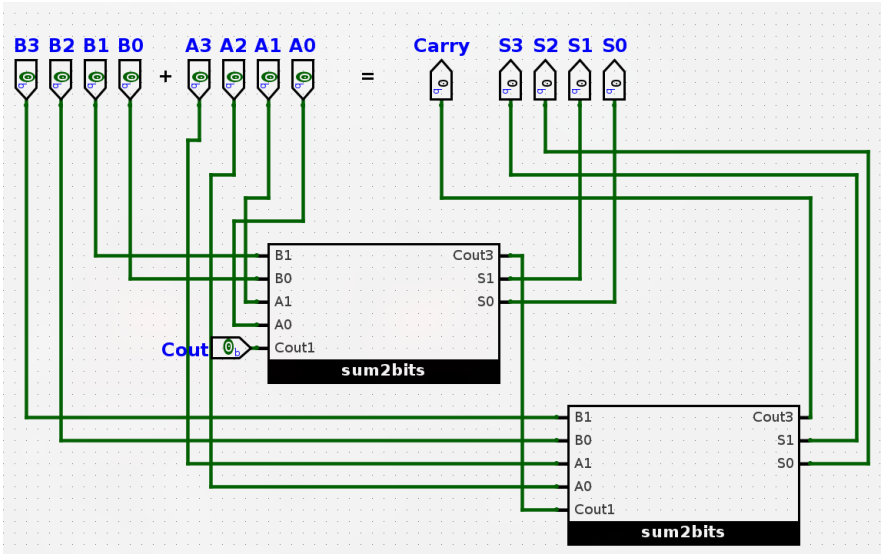

Figura 8: Sumador de 4 bits

2.2.4. Sumador de 8 bits

Despues se implementó el sumador de 8 bits conectando dos sumadores de 4 bits(ocho sumadores de 1 bit). En este caso, se pueden sumar dos números de 8 bits (A y B), propagando el acarreo a lo largo de todos los bloques.

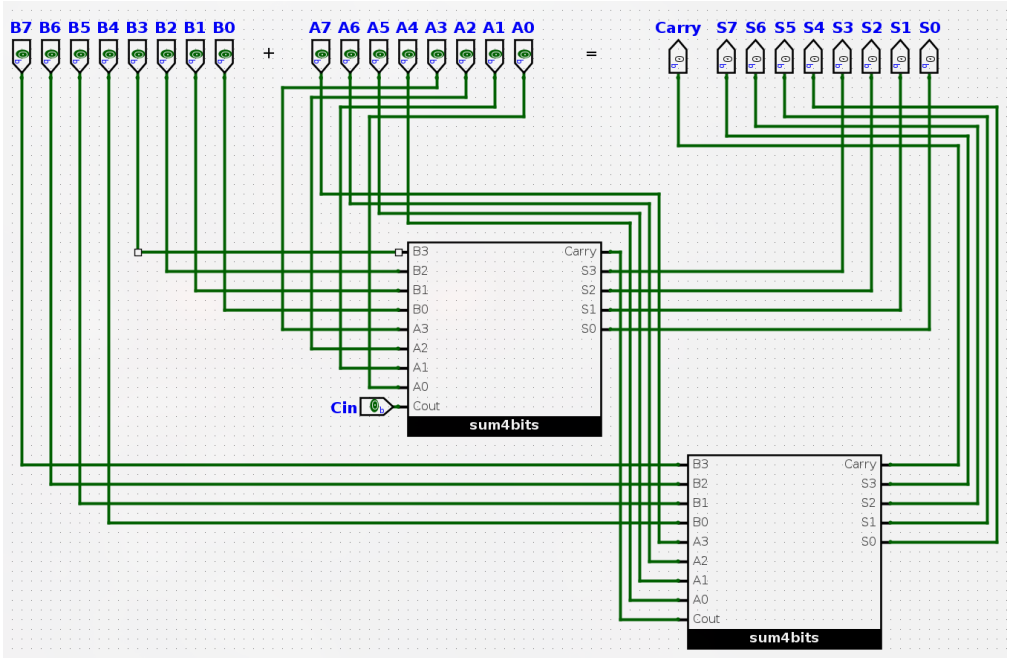

Figura 9: Sumador de 8 bits construido a partir de Full Adders

2.2.5. Refactorizar sumador de 8 bits

Finalmente se organizaron las entradas de 8 bits para tener un circuito mas sencillo de entender. Este circuito (SUMA_op) se usa mas adelante para las operaciones de restar y multiplicar.

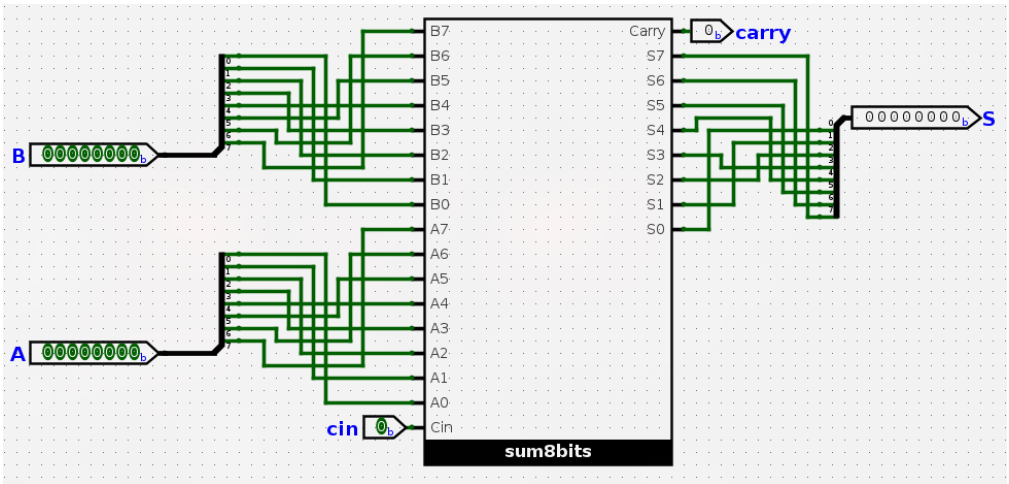

Figura 10: Sumador de 8 bits con 3 entradas y dos salidas

Este diseño permite observar claramente cómo el acarreo se propaga entre cada etapa, lo cual es clave para entender el funcionamiento de los sumadores en sistemas digitales.

2.3. Restador de 8 bits

El restador se implementó usando el mismo sumador de 8 bits como se menciono antes, aplicando el método de complemento a dos.

Primero se invirtieron los bits de B y luego se sumó 1. Después, este resultado se sumó con A. Para esto, se configuró el sumador con la entrada B invertida reutilizando la operacion logica previamente fabricada antes (NOT_op) y el acarreo inicial en 1.

Esto permitió reutilizar el circuito del sumador y el circuito not sin necesidad de diseñar uno nuevo.

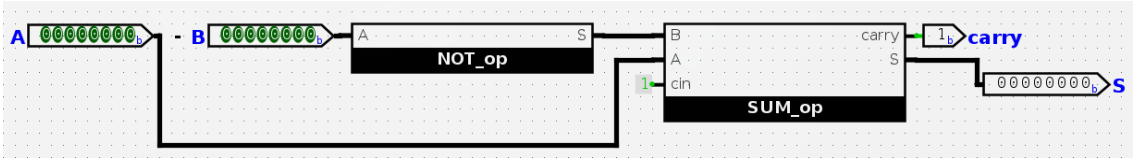

Figura 11: Implementación del restador usando complemento a dos

2.4. Multiplicador de 8 bits

El multiplicador se diseñó dividiendo el problema en varios subcircuitos, con el fin de hacerlo más claro y modular. La idea general es generar productos parciales y luego sumarlos.

2.4.1. Producto de 8 bits por 1 bit

Primero se diseñó un circuito que multiplica un número de 8 bits por un solo bit.

Para esto, se toma la entrada A de 8 bits y un bit de B. Este bit se convierte en un bus de 8 bits usando un separador, de forma que:

- Si B = 0, se obtiene 00000000
- Si B = 1, se obtiene 11111111

Luego, se aplica una compuerta AND bit a bit entre A y este nuevo bus. Esto permite obtener el producto entre A y un solo bit.

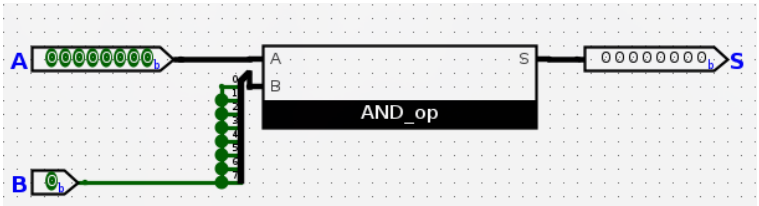

Figura 12: Multiplicación de un número de 8 bits por 1 bit

2.4.2. Generador de productos parciales

Después, este bloque se reutilizó 8 veces. Cada uno recibe la misma entrada A, pero un bit diferente de B (desde b_0 hasta b_7).

Cada bloque genera un resultado distinto ($S_0, S_1, ..., S_7$), que corresponde a los productos parciales de la multiplicación.

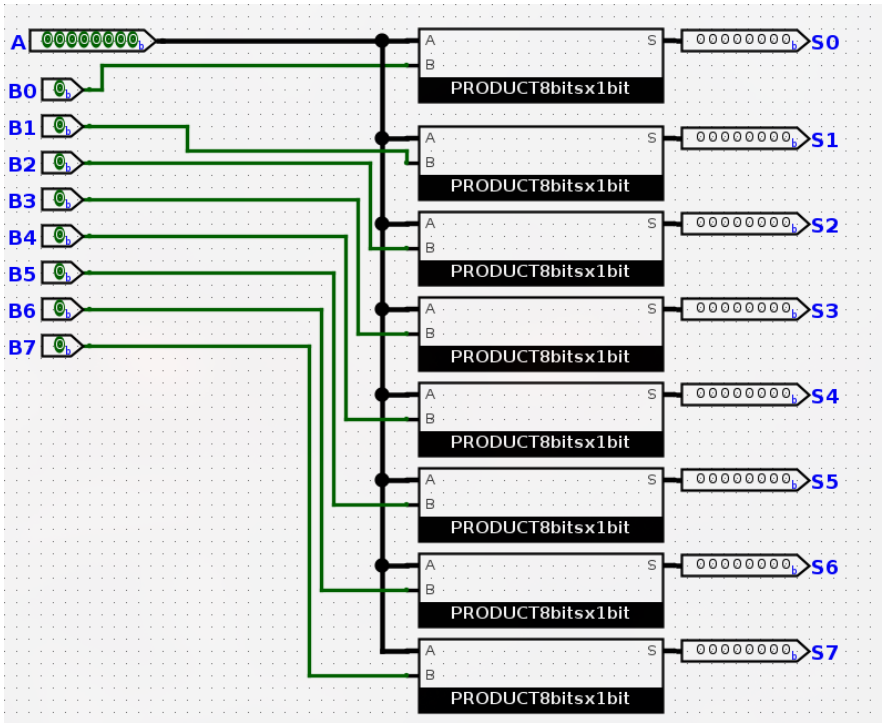

Figura 13: Generación de productos parciales

2.4.3. Desplazamiento a la izquierda

Cada producto parcial debe desplazarse hacia la izquierda según su posición:

- S_0: no se desplaza
- S_1: se desplaza 1 bit
- S_2: se desplaza 2 bits
- ...
- S_7: se desplaza 7 bits

Esto se implementó usando las conexiones con los separadores, usando las constantes 0 en los bits que se movieron.

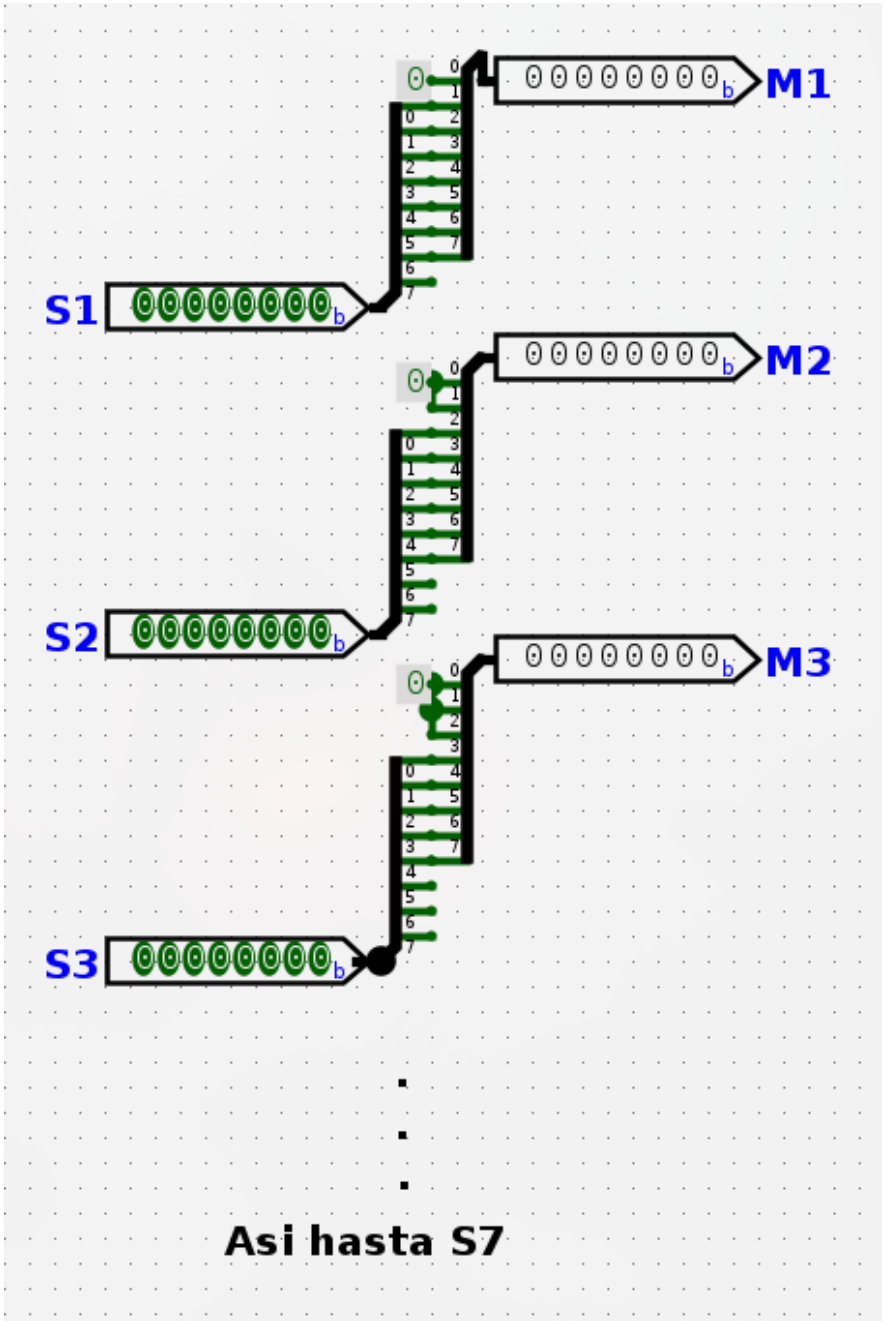

Figura 14: Desplazamiento de productos parciales

2.4.4. Suma de productos parciales

Finalmente, todos los resultados se suman utilizando un sumador en cascada. Primero se suma:

$$R_1 = S_0 + M_1$$

Luego:

$$R_2 = R_1 + M_2$$

Y así sucesivamente hasta sumar todos los términos desplazados.

Este proceso acumula todos los productos parciales en un solo resultado final.

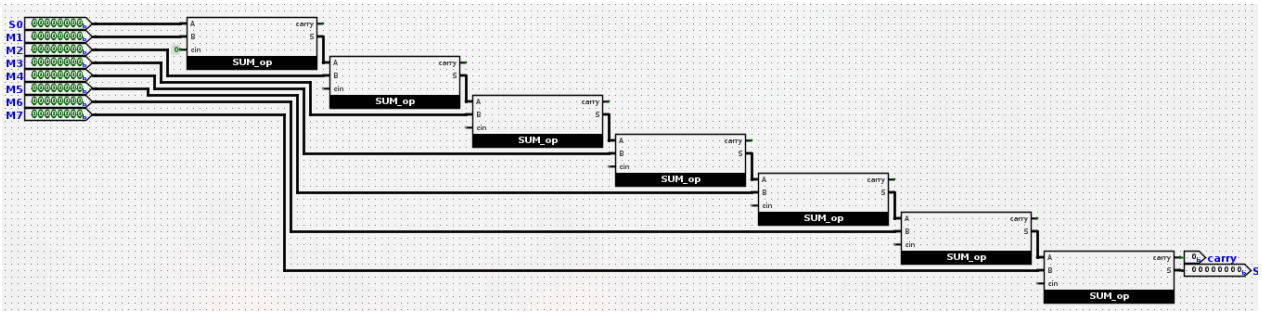

Figura 15: Suma en cascada de los productos parciales

2.4.5. Circuito final de multiplicación

El circuito completo (PRODUCT_op)recibe dos entradas de 8 bits (A y B), genera los productos parciales, los desplaza y finalmente los suma.

Debido a que el resultado puede ser de hasta 16 bits, en esta práctica solo se toman los 8 bits menos significativos.

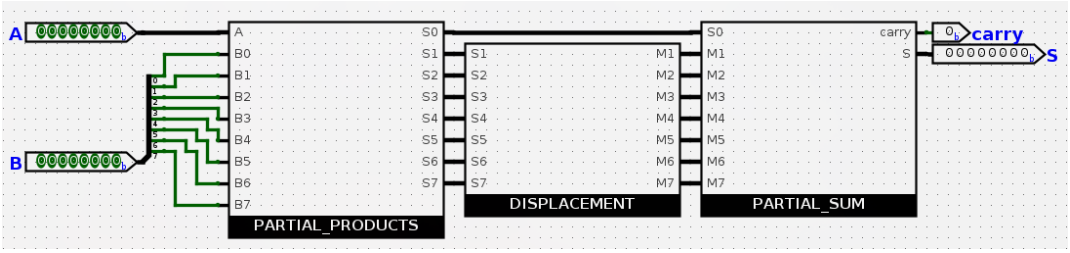

Figura 16: Multiplicador de 8 bits completo

2.5. Multiplexor de 8 a 1

El multiplexor se construyó a partir de un MUX de 1 bit. Luego, se copiaron 8 de estos para manejar un bus de 8 bits.

Cada entrada del multiplexor corresponde a una operación diferente (suma, resta, AND, OR, etc.), y las líneas de selección (Opcode) determinan cuál se envía a la salida.

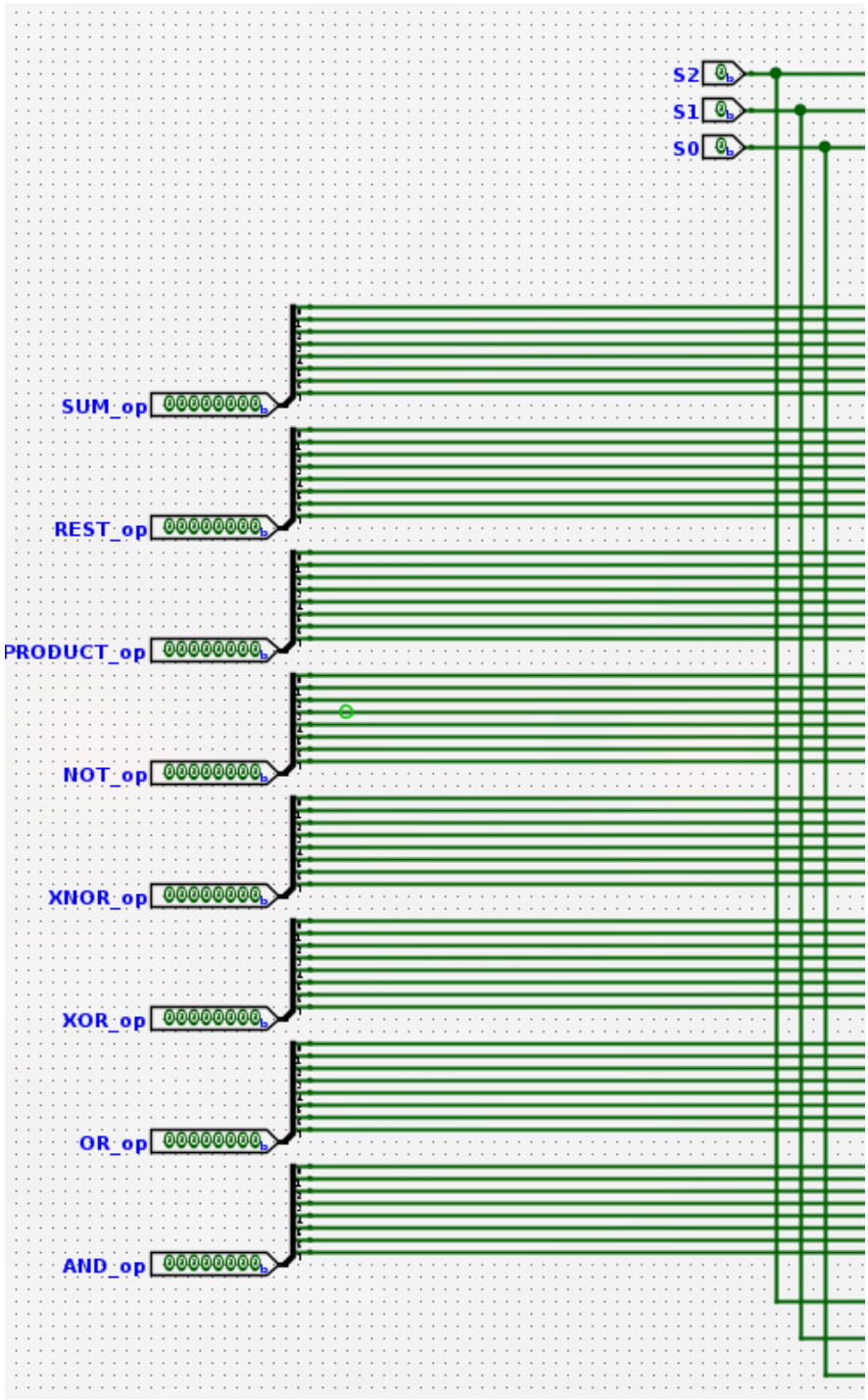

Figura 17: Entradas de Multpiplexor de 8 a 1

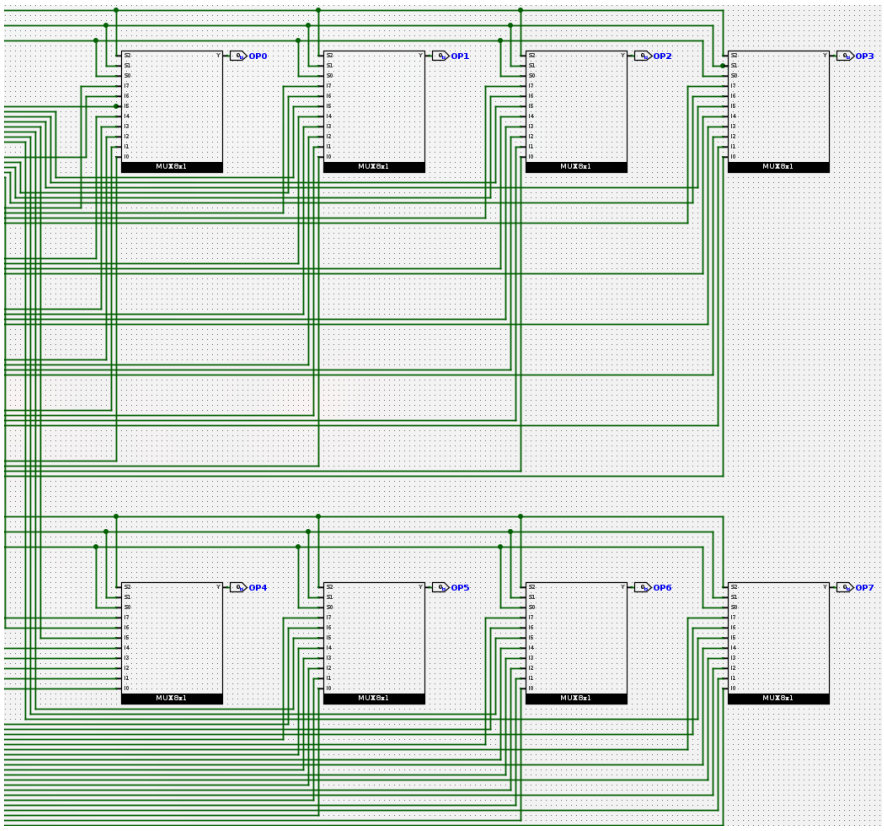

Figura 18: Multiplexor de 8 a 1 para 8 bits

2.6. Integración de la ALU

Finalmente, todos los módulos se conectaron en un solo circuito. Las entradas A y B se conectan a todos los bloques al mismo tiempo, y cada uno genera su resultado en paralelo.

Luego, el multiplexor selecciona cuál de esos resultados será la salida final, según el valor del Opcode. Tambien se agrego un Enable reutilizando el circuito AND_op para poder activar o desactivar la Lua de manera informal.

Este proceso permitió obtener una ALU completamente funcional, capaz de realizar diferentes operaciones dependiendo de la señal de control.

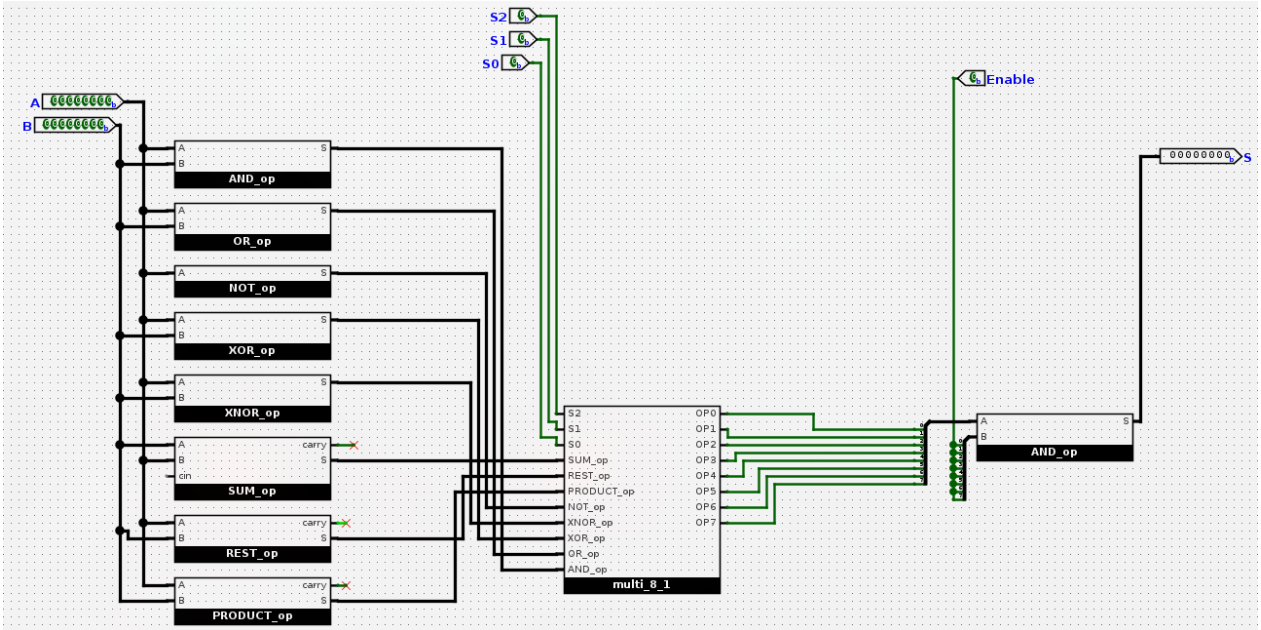

Figura 19: ALU de 8 bits completa en Logisim

3. Resultados

En esta sección se muestran los resultados obtenidos al simular la ALU en Logisim. Se realizaron diferentes pruebas para cada operación, verificando que la salida corresponde correctamente según el opcode seleccionado.

Para cada caso se presentan las entradas, el opcode utilizado y la salida obtenida.

3.1. Operaciones lógicas

3.1.1. AND (Opcode 000)

- A = 10101010
- B = 11001100
- Resultado esperado: 10001000

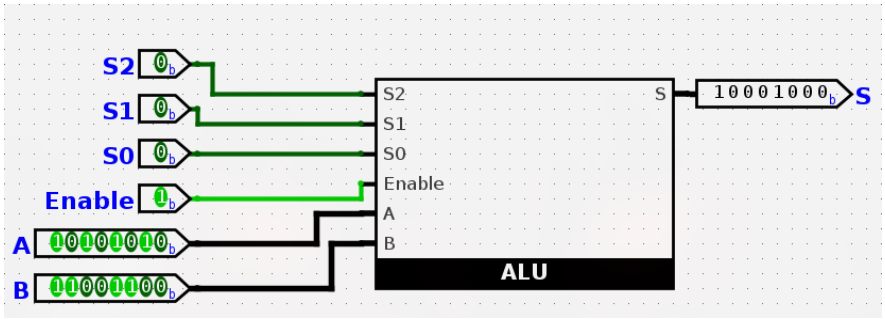

Figura 20: Prueba de la operación AND

3.1.2. OR (Opcode 001)

- A = 10101010
- B = 11001100
- Resultado esperado: 11101110

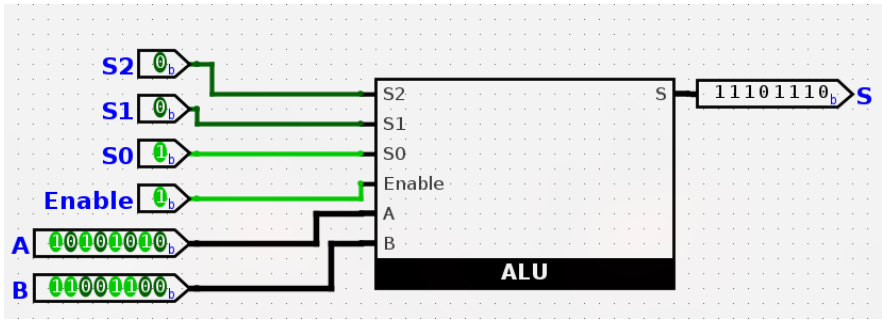

Figura 21: Prueba de la operación OR

3.1.3. XOR (Opcode 010)

- A = 10101010
- B = 11001100
- Resultado esperado: 01100110

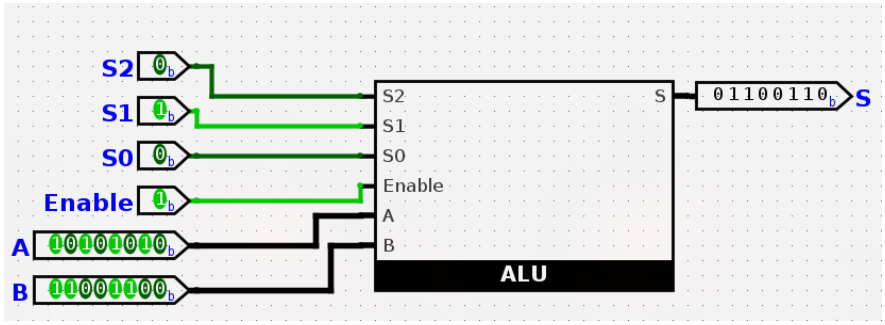

Figura 22: Prueba de la operación XOR

3.1.4. XNOR (Opcode 011)

- A = 10101010
- B = 11001100
- Resultado esperado: 10011001

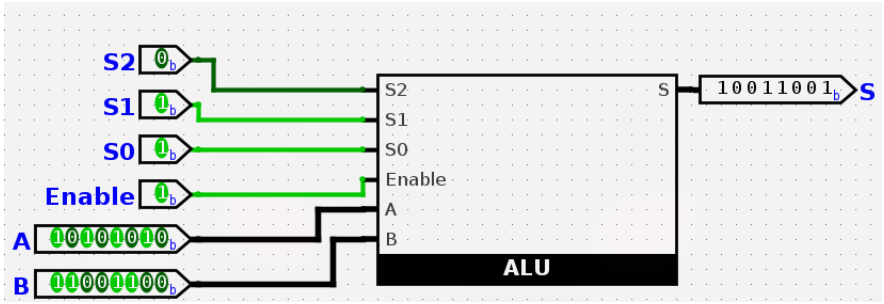

Figura 23: Prueba de la operación XNOR

3.1.5. NOT (Opcode 100)

- A = 10101010
- Resultado esperado: 01010101

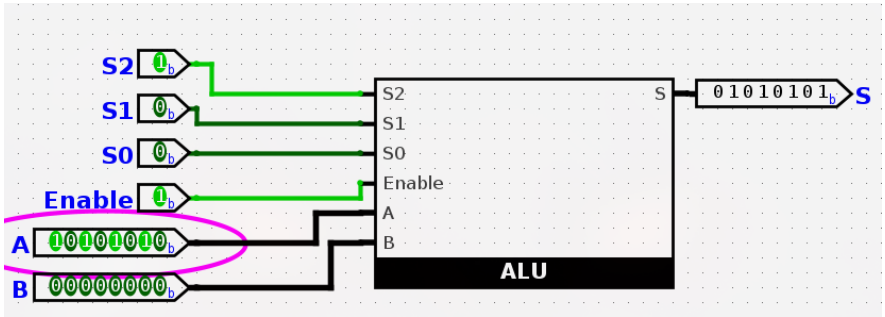

Figura 24: Prueba de la operación NOT

3.2. Operaciones aritméticas

3.2.1. Suma (Opcode 101)

- A = 00000101 (5)
- B = 00000011 (3)
- Resultado esperado: 00001000 (8)

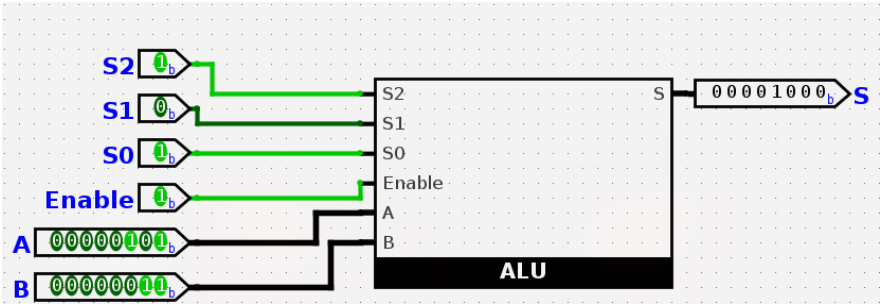

Figura 25: Prueba de la operación de suma

3.2.2. Resta (Opcode 110)

- A = 00000101 (5)
- B = 00000011 (3)
- Resultado esperado: 00000010 (2)

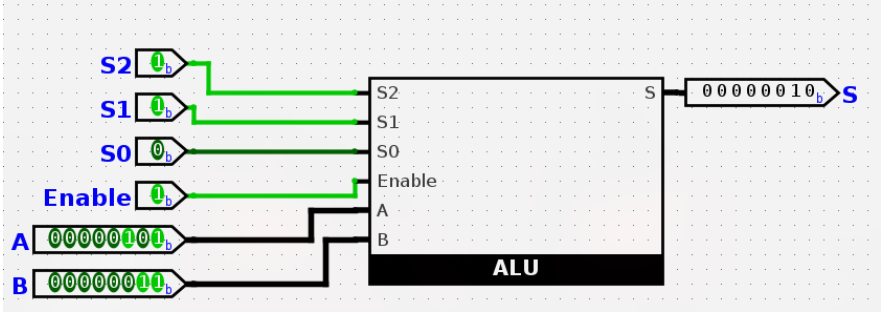

Figura 26: Prueba de la operación de resta

3.2.3. Multiplicación (Opcode 111)

- A = 00000011 (3)
- B = 00000100 (4)
- Resultado esperado: 00001100 (12)

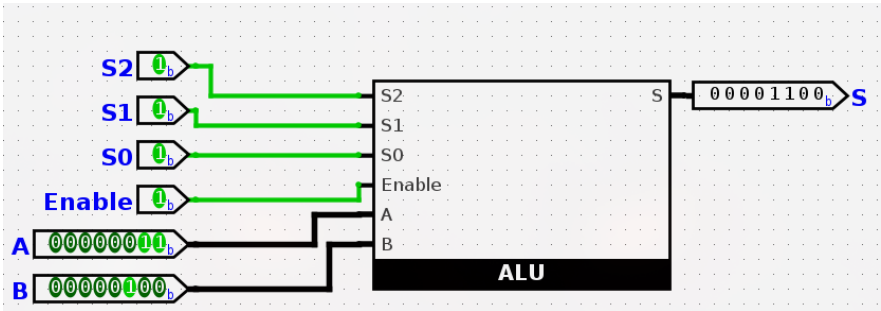

Figura 27: Prueba de la operación de multiplicación

3.3. Observaciones

En todas las pruebas realizadas, la ALU respondió correctamente según el opcode seleccionado. Las salidas coinciden con los resultados esperados, lo que confirma que el diseño y la integración de los módulos fueron adecuados.

También se verificó que el cambio de operación se realiza correctamente al modificar las líneas de selección, sin interferencias entre los distintos bloques.

4. Análisis y discusión

Durante el desarrollo de la ALU se pudo entender mejor cómo se construyen **sistemas más complejos a partir de circuitos básicos**. Dividir el problema en **módulos** (operaciones lógicas, sumador, restador, multiplicador y multiplexor) ayudó bastante a organizar el trabajo y a detectar errores más fácilmente.

Una de las **principales dificultades** fue el manejo del **acarreo en el sumador**, ya que si una conexión no estaba bien hecha, todo el resultado se veía afectado. Esto se solucionó revisando paso a paso la **propagación del carry** entre cada bloque.

El multiplicador fue la parte más compleja, ya que implicó trabajar con varios subcircuitos (**productos parciales, desplazamientos y suma en cascada**). Para hacerlo más entendible, se dividió en bloques más pequeños, lo que facilitó tanto su diseño como su verificación.

También se compararon diferentes formas de implementar algunos módulos. Por ejemplo, el restador pudo haberse hecho con un diseño independiente, pero **usar el sumador simplifica mucho el circuito**. En el caso del multiplicador, aunque existen formas más optimizadas, el **método de productos parciales** permite entender mejor el proceso.

En cuanto al rendimiento, el circuito cumple correctamente con todas las operaciones, aunque aun no se tiene las flags para determinar si hay un carry o borrow en la operacion final. Se desarrollo otra LUA con circuitos independientes para verificar las banderas como prueba.

5. Conclusiones

En esta práctica se logró diseñar e implementar una **ALU de 8 bits** utilizando únicamente compuertas lógicas y circuitos combinacionales. Esto permitió comprender de manera más clara **cómo funcionan internamente las operaciones** que normalmente se dan por sentadas en los computadores.

Uno de los **aprendizajes más importantes** fue entender cómo se pueden construir **operaciones complejas a partir de bloques simples**, como el caso del sumador y su reutilización para la resta. También se reforzó el **concepto de trabajo modular**, que facilita mucho el diseño de sistemas grandes.

Además, se evidenció la **importancia de los circuitos combinacionales** en el desarrollo de sistemas digitales, ya que estos son la base de componentes más avanzados dentro de un computador.

Como recomendación, sería útil en futuras prácticas explorar **formas más optimizadas**, especialmente en el multiplicador. También se podría trabajar con **más bits o agregar nuevas operaciones** a la ALU para ampliar su funcionalidad.

6. Enlace a la carpeta compartida

A continuación se presenta el enlace a la carpeta donde se encuentran los archivos del proyecto:

`https://github.com/iYeremy/ALU_desing/releases/tag/v1.0.0`

7. Referencias

- **Mano, M. M., & Ciletti, M. D.** (2017). *Digital Design* (6th ed.). Pearson.
- **Logisim Evolution**. (n.d.). *Open-source digital logic simulator*. Recuperado de: `https://github.com/logisim-evolution/logisim-evolution`